

Министерство науки и высшего образования РФ
Пензенский государственный университет
Кафедра «Вычислительная техника»

ОТЧЁТ

По лабораторной работе №6

По дисциплине «Логика и основы алгоритмизации в инженерных
задачах»

На тему «Унарные и бинарные операции над графами»

**Выполнил студент группы
24ВВВ2:**

Тремасов К. И.

Приняли:

Юрова О.В.

Деев М.В.

Пенза, 2025

Цель работы

Научиться реализовывать алгоритм для выполнения унарных и бинарных операций над графами.

Лабораторное задание

Задание 1

1. Сгенерировать (используя генератор случайных чисел) две матрицы M_1 , M_2 смежности неориентированных помеченных графов G_1 , G_2 . Вывести сгенерированные матрицы на экран.
2. Для указанных графов преобразовать представление матриц смежности в списки смежности. Вывести полученные списки на экран.

Задание 2

1. Для матричной формы представления графов выполнить операцию:
 - а) отождествления вершин
 - б) стягивания ребра
 - в) расщепления вершиныНомера выбираемых для выполнения операции вершин ввести с клавиатуры. Результат выполнения операции вывести на экран.
2. Для представления графов в виде списков смежности выполнить операцию:
 - а) отождествления вершин
 - б) стягивания ребра
 - в) расщепления вершиныНомера выбираемых для выполнения операции вершин ввести с клавиатуры. Результат выполнения операции вывести на экран.

Задание 3

1. Для матричной формы представления графов выполнить операцию:
 - а) объединения $G = G_1 \cup G_2$
 - б) пересечения $G = G_1 \cap G_2$
 - в) кольцевой суммы $G = G_1 \oplus G_2$

Результат выполнения операции вывести на экран.

Задание 4

1. Для матричной формы представления графов выполнить операцию декартова произведения графов $G = G_1 \times G_2$. Результат выполнения операции вывести на экран.

Описание метода решения задачи

Для реализации операций над графами используются два представления: матрица смежности и списки смежности. В подготовительном этапе генерируются случайные графы заданного размера с вероятностным формированием рёбер и петель. При выполнении унарных операций над матрицей алгоритм физически модифицирует двумерный массив: при отождествлении вершин связи удаляемой вершины переносятся в сохраняемую, затем осуществляется сдвиг строк и столбцов для устранения пустого места; при расщеплении вершины размер матрицы увеличивается, связи исходной вершины копируются в новую, после чего они связываются дополнительным ребром. Для списков смежности операции работают через манипуляции с указателями: создание и удаление узлов, перелинковка списков, сдвиг массива указателей. Бинарные операции над матрицами реализуются через поэлементные логические преобразования - объединение использует ИЛИ, пересечение работает через И, кольцевая сумма применяет исключающее ИЛИ, а декартово произведение строит новую матрицу по комбинаторным правилам связывания пар вершин. Все операции сохраняют свойство неориентированности графов через симметричное заполнение матриц и двустороннее добавление связей в списках.

Листинг программы

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <locale.h>
```

```

#define MAX_VERTICES 20

#define _CRT_SECURE_NO_WARNINGS

typedef struct Node {
    int vertex;
    struct Node* next;
} Node;

typedef struct {
    int matrix[MAX_VERTICES][MAX_VERTICES];
    int num_vertices;
} GraphMatrix;

typedef struct {
    Node* adjacency_list[MAX_VERTICES];
    int num_vertices;
} GraphList;

void initRandom() {
    srand((unsigned int)time(NULL));
}

void copyGraph(GraphMatrix* dest, GraphMatrix* src) {
    dest->num_vertices = src->num_vertices;
    for (int i = 0; i < src->num_vertices; i++) {
        for (int j = 0; j < src->num_vertices; j++) {
            dest->matrix[i][j] = src->matrix[i][j];
        }
    }
}

void freeGraphListVertex(GraphList* graph, int vertex);

void generateRandomGraph(GraphMatrix* graph, int vertices) {

```

```

graph->num_vertices = vertices;

for (int i = 0; i < vertices; i++) {
    for (int j = 0; j < vertices; j++) {
        if (i == j) {
            graph->matrix[i][j] = (rand() % 100) < 30 ? 1 : 0;
        }
        else if (i < j) {
            int value = (rand() % 100) < 50 ? 1 : 0;
            graph->matrix[i][j] = value;
            graph->matrix[j][i] = value;
        }
    }
}

void printMatrix(GraphMatrix* graph) {
    printf("Матрица смежности (%d вершин):\n", graph->num_vertices);
    printf(" ");
    for (int i = 0; i < graph->num_vertices; i++) {
        printf("%2d ", i);
    }
    printf("\n");

    for (int i = 0; i < graph->num_vertices; i++) {
        printf("%2d ", i);
        for (int j = 0; j < graph->num_vertices; j++) {
            printf("%2d ", graph->matrix[i][j]);
        }
        printf("\n");
    }
    printf("\n");
}

```

```

Node* createNode(int vertex) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (newNode == NULL) {
        return NULL;
    }
    newNode->vertex = vertex;
    newNode->next = NULL;
    return newNode;
}

void matrixToList(GraphMatrix* matrixGraph, GraphList* listGraph) {
    listGraph->num_vertices = matrixGraph->num_vertices;

    for (int i = 0; i < listGraph->num_vertices; i++) {
        listGraph->adjacency_list[i] = NULL;
    }

    for (int i = 0; i < matrixGraph->num_vertices; i++) {
        for (int j = 0; j < matrixGraph->num_vertices; j++) {
            if (matrixGraph->matrix[i][j] == 1) {
                Node* newNode = createNode(j);
                if (newNode != NULL) {
                    newNode->next = listGraph->adjacency_list[i];
                    listGraph->adjacency_list[i] = newNode;
                }
            }
        }
    }
}

void printList(GraphList* graph) {
    printf("Список смежности:\n");

```

```

for (int i = 0; i < graph->num_vertices; i++) {
    printf("Вершина %d: ", i);
    Node* current = graph->adjacency_list[i];
    while (current != NULL) {
        printf("%d ", current->vertex);
        current = current->next;
    }
    printf("\n");
}
printf("\n");
}

void identifyVerticesMatrix(GraphMatrix* graph, int v1, int v2) {
    if (v1 == v2 || v1 >= graph->num_vertices || v2 >= graph->num_vertices) {
        printf("Некорректные вершины\n");
        return;
    }

    int keep_vertex = (v1 < v2) ? v1 : v2;
    int remove_vertex = (v1 < v2) ? v2 : v1;

    for (int i = 0; i < graph->num_vertices; i++) {
        if (i != keep_vertex && i != remove_vertex) {
            if (graph->matrix[remove_vertex][i] == 1) {
                graph->matrix[keep_vertex][i] = 1;
                graph->matrix[i][keep_vertex] = 1;
            }
        }
    }

    if (graph->matrix[remove_vertex][remove_vertex] == 1) {
        graph->matrix[keep_vertex][keep_vertex] = 1;
    }
}

```

```

    for (int i = remove_vertex; i < graph->num_vertices - 1; i++) {
        for (int j = 0; j < graph->num_vertices; j++) {
            graph->matrix[i][j] = graph->matrix[i + 1][j];
        }
    }

    for (int i = 0; i < graph->num_vertices - 1; i++) {
        for (int j = remove_vertex; j < graph->num_vertices - 1; j++) {
            graph->matrix[i][j] = graph->matrix[i][j + 1];
        }
    }

    graph->num_vertices--;
}

void contractEdgeMatrix(GraphMatrix* graph, int v1, int v2) {
    if (v1 == v2 || v1 >= graph->num_vertices || v2 >= graph->num_vertices) {
        printf("Некорректные вершины\n");
        return;
    }

    if (graph->matrix[v1][v2] == 0) {
        printf("Ребро между вершинами %d и %d не существует\n", v1, v2);
        return;
    }

    identifyVerticesMatrix(graph, v1, v2);
}

void splitVertexMatrix(GraphMatrix* graph, int v) {
    if (v >= graph->num_vertices) {
        printf("Некорректная вершина\n");
    }
}

```



```

        return;
    }

    if (graph->num_vertices >= MAX_VERTICES) {
        printf("Достигнуто максимальное количество вершин\n");
        return;
    }

    int old_vertices = graph->num_vertices;
    graph->num_vertices++;
    int newVertex = graph->num_vertices - 1;

    for (int i = 0; i < graph->num_vertices; i++) {
        graph->matrix[newVertex][i] = 0;
        graph->matrix[i][newVertex] = 0;
    }

    for (int i = 0; i < old_vertices; i++) {
        if (graph->matrix[v][i] == 1) {
            if (i == v) {

                graph->matrix[newVertex][newVertex] = 1;
            }
            else {

                graph->matrix[newVertex][i] = 1;
                graph->matrix[i][newVertex] = 1;
            }
        }
    }

    graph->matrix[v][newVertex] = 1;
    graph->matrix[newVertex][v] = 1;

```

```
}
```

```
void identifyVerticesList(GraphList* graph, int v1, int v2) {  
    if (v1 == v2 || v1 >= graph->num_vertices || v2 >= graph->num_vertices) {  
        printf("Некорректные вершины\n");  
        return;  
    }  
}
```

```
int keep_vertex = (v1 < v2) ? v1 : v2;  
int remove_vertex = (v1 < v2) ? v2 : v1;
```

```
Node* current = graph->adjacency_list[remove_vertex];  
while (current != NULL) {  
    if (current->vertex != keep_vertex && current->vertex !=  
remove_vertex) {  
        int exists = 0;  
        Node* check = graph->adjacency_list[keep_vertex];  
        while (check != NULL) {  
            if (check->vertex == current->vertex) {  
                exists = 1;  
                break;  
            }  
            check = check->next;  
        }  
    }  
}
```

```
if (!exists) {  
    Node* newNode = createNode(current->vertex);  
    newNode->next = graph->adjacency_list[keep_vertex];  
    graph->adjacency_list[keep_vertex] = newNode;  
  
    Node* reverseNode = createNode(keep_vertex);  
    reverseNode->next = graph->adjacency_list[current->vertex];  
    graph->adjacency_list[current->vertex] = reverseNode;
```

```

        }
    }
    current = current->next;
}

for (int i = 0; i < graph->num_vertices; i++) {
    Node* prev = NULL;
    Node* curr = graph->adjacency_list[i];

    while (curr != NULL) {
        if (curr->vertex == remove_vertex) {
            if (prev == NULL) {
                graph->adjacency_list[i] = curr->next;
                free(curr);
                curr = graph->adjacency_list[i];
            }
            else {
                prev->next = curr->next;
                free(curr);
                curr = prev->next;
            }
        }
        else {
            prev = curr;
            curr = curr->next;
        }
    }
}

```

```

for (int i = 0; i < graph->num_vertices; i++) {
    Node* curr = graph->adjacency_list[i];
    while (curr != NULL) {
        if (curr->vertex > remove_vertex) {

```

```

        curr->vertex--;
    }
    curr = curr->next;
}
}

for (int i = remove_vertex; i < graph->num_vertices - 1; i++) {
    graph->adjacency_list[i] = graph->adjacency_list[i + 1];
}
graph->adjacency_list[graph->num_vertices - 1] = NULL;
graph->num_vertices--;
}

void contractEdgeList(GraphList* graph, int v1, int v2) {
    if (v1 == v2 || v1 >= graph->num_vertices || v2 >= graph->num_vertices) {
        printf("Некорректные вершины\n");
        return;
    }

    int edge_exists = 0;
    Node* current = graph->adjacency_list[v1];
    while (current != NULL) {
        if (current->vertex == v2) {
            edge_exists = 1;
            break;
        }
        current = current->next;
    }

    if (!edge_exists) {
        printf("Ребро между вершинами %d и %d не существует!\n", v1, v2);
        printf("Текущие списки для проверки:\n");
        printList(graph);
    }
}

```

```

        return;
    }

    printf("Стягивание ребра между %d и %d\n", v1, v2);
    identifyVerticesList(graph, v1, v2);
}

void splitVertexList(GraphList* graph, int v) {
    if (v >= graph->num_vertices) {
        printf("Некорректная вершина!\n");
        return;
    }

    if (graph->num_vertices >= MAX_VERTICES) {
        printf("Достигнуто максимальное количество вершин\n");
        return;
    }

    printf("Расщепление вершины %d\n", v);

    graph->num_vertices++;
    int newVertex = graph->num_vertices - 1;
    graph->adjacency_list[newVertex] = NULL;

    Node* temp_list = NULL;
    Node* current = graph->adjacency_list[v];

    while (current != NULL) {
        Node* temp_node = createNode(current->vertex);
        if (temp_node != NULL) {
            temp_node->next = temp_list;
            temp_list = temp_node;
        }
    }
}

```

```

        current = current->next;
    }

    current = temp_list;
    while (current != NULL) {
        if (current->vertex == v) {
            Node* loop_node = createNode(newVertex);
            loop_node->next = graph->adjacency_list[newVertex];
            graph->adjacency_list[newVertex] = loop_node;
        }
        else {
            Node* new_node = createNode(current->vertex);
            new_node->next = graph->adjacency_list[newVertex];
            graph->adjacency_list[newVertex] = new_node;

            Node* reverse_node = createNode(newVertex);
            reverse_node->next = graph->adjacency_list[current->vertex];
            graph->adjacency_list[current->vertex] = reverse_node;
        }
        current = current->next;
    }

    Node* link1 = createNode(newVertex);
    link1->next = graph->adjacency_list[v];
    graph->adjacency_list[v] = link1;

    Node* link2 = createNode(v);
    link2->next = graph->adjacency_list[newVertex];
    graph->adjacency_list[newVertex] = link2;

    while (temp_list != NULL) {
        Node* temp = temp_list;
        temp_list = temp_list->next;
    }

```

```

        free(temp);
    }

    printf("После расщепления вершины %d добавлена вершина %d\n", v,
newVertex);
}

void freeGraphListVertex(GraphList* graph, int vertex) {
    Node* current = graph->adjacency_list[vertex];
    while (current != NULL) {
        Node* temp = current;
        current = current->next;
        free(temp);
    }
    graph->adjacency_list[vertex] = NULL;
}

void unionGraphs(GraphMatrix* g1, GraphMatrix* g2, GraphMatrix* result) {
    int max_vertices = (g1->num_vertices > g2->num_vertices) ? g1-
>num_vertices : g2->num_vertices;
    result->num_vertices = max_vertices;

    for (int i = 0; i < max_vertices; i++) {
        for (int j = 0; j < max_vertices; j++) {
            int val1 = (i < g1->num_vertices && j < g1->num_vertices) ? g1-
>matrix[i][j] : 0;
            int val2 = (i < g2->num_vertices && j < g2->num_vertices) ? g2-
>matrix[i][j] : 0;
            result->matrix[i][j] = val1 || val2;
        }
    }
}

void intersectGraphs(GraphMatrix* g1, GraphMatrix* g2, GraphMatrix* result) {

```

```

    int min_vertices = (g1->num_vertices < g2->num_vertices) ? g1-
>num_vertices : g2->num_vertices;

    result->num_vertices = min_vertices;

    for (int i = 0; i < min_vertices; i++) {
        for (int j = 0; j < min_vertices; j++) {
            result->matrix[i][j] = g1->matrix[i][j] && g2->matrix[i][j];
        }
    }
}

```

```

void ringSumGraphs(GraphMatrix* g1, GraphMatrix* g2, GraphMatrix* result) {
    int max_vertices = (g1->num_vertices > g2->num_vertices) ? g1-
>num_vertices : g2->num_vertices;

    result->num_vertices = max_vertices;

    for (int i = 0; i < max_vertices; i++) {
        for (int j = 0; j < max_vertices; j++) {
            int val1 = (i < g1->num_vertices && j < g1->num_vertices) ? g1-
>matrix[i][j] : 0;
            int val2 = (i < g2->num_vertices && j < g2->num_vertices) ? g2-
>matrix[i][j] : 0;
            result->matrix[i][j] = val1 != val2;
        }
    }
}

```

```

void cartesianProduct(GraphMatrix* g1, GraphMatrix* g2, GraphMatrix* result)
{
    int n = g1->num_vertices;
    int m = g2->num_vertices;
    result->num_vertices = n * m;

    for (int i = 0; i < result->num_vertices; i++) {
        for (int j = 0; j < result->num_vertices; j++) {

```



```

        while (current != NULL) {
            Node* temp = current;
            current = current->next;
            free(temp);
        }
    }
}

int main() {
    setlocale(LC_ALL, "Russian");
    initRandom();

    GraphMatrix M1, M2;
    GraphList L1, L2;

    int vertices1, vertices2;

    printf("ЗАДАНИЕ 1\n");
    printf("Введите количество вершин для графа G1: ");
    scanf_s("%d", &vertices1);
    printf("Введите количество вершин для графа G2: ");
    scanf_s("%d", &vertices2);

    if (vertices1 > MAX_VERTICES || vertices1 <= 0) {
        printf("Некорректное количество вершин для G1! Используется значение по умолчанию: 5\n");
        vertices1 = 5;
    }

    if (vertices2 > MAX_VERTICES || vertices2 <= 0) {
        printf("Некорректное количество вершин для G2! Используется значение по умолчанию: 5\n");
        vertices2 = 5;
    }
}

```

```
generateRandomGraph(&M1, vertices1);
generateRandomGraph(&M2, vertices2);

printf("Граф G1 (%d вершин):\n", vertices1);
printMatrix(&M1);

printf("Граф G2 (%d вершин):\n", vertices2);
printMatrix(&M2);

matrixToList(&M1, &L1);
matrixToList(&M2, &L2);

printf("Список смежности G1:\n");
printList(&L1);

printf("Список смежности G2:\n");
printList(&L2);

printf("ЗАДАНИЕ 2\n");

int v1, v2, v;
GraphMatrix current_graph1, current_graph2;

printf("\nОперации над матрицами\n");

printf("Граф G1\n");

copyGraph(&current_graph1, &M1);

printf("Введите вершины для отождествления (v1 v2): ");
scanf_s("%d %d", &v1, &v2);
identifyVerticesMatrix(&current_graph1, v1, v2);
printf("G1 после отождествления:\n");
```

```
printMatrix(&current_graph1);

printf("Введите ребро для стягивания (v1 v2): ");
scanf_s("%d %d", &v1, &v2);
contractEdgeMatrix(&current_graph1, v1, v2);
printf("G1 после стягивания:\n");
printMatrix(&current_graph1);

printf("Введите вершину для расщепления: ");
scanf_s("%d", &v);
splitVertexMatrix(&current_graph1, v);
printf("G1 после расщепления:\n");
printMatrix(&current_graph1);

printf("\nГраф G2\n");

copyGraph(&current_graph2, &M2);

printf("G2 после отождествления вершин %d и %d:\n", v1, v2);
identifyVerticesMatrix(&current_graph2, v1, v2);
printMatrix(&current_graph2);

printf("G2 после стягивания ребра между %d и %d:\n", v1, v2);
contractEdgeMatrix(&current_graph2, v1, v2);
printMatrix(&current_graph2);

printf("G2 после расщепления вершины %d:\n", v);
splitVertexMatrix(&current_graph2, v);
printMatrix(&current_graph2);

printf("\nОперации над списками\n");

printf("Списки G1\n");
```

```
GraphList L1_copy = L1;
```

```
printf("Введите вершины для отождествления (v1 v2): ");
```

```
scanf_s("%d %d", &v1, &v2);
```

```
identifyVerticesList(&L1_copy, v1, v2);
```

```
printf("G1 после отождествления:\n");
```

```
printList(&L1_copy);
```

```
printf("Введите ребро для стягивания (v1 v2): ");
```

```
scanf_s("%d %d", &v1, &v2);
```

```
contractEdgeList(&L1_copy, v1, v2);
```

```
printf("G1 после стягивания:\n");
```

```
printList(&L1_copy);
```

```
printf("Введите вершину для расщепления: ");
```

```
scanf_s("%d", &v);
```

```
splitVertexList(&L1_copy, v);
```

```
printf("G1 после расщепления:\n");
```

```
printList(&L1_copy);
```

```
printf("\nСписки G2\n");
```

```
GraphList L2_copy = L2;
```

```
printf("G2 после отождествления вершин %d и %d:\n", v1, v2);
```

```
identifyVerticesList(&L2_copy, v1, v2);
```

```
printList(&L2_copy);
```

```
printf("G2 после стягивания ребра между %d и %d:\n", v1, v2);
```

```
contractEdgeList(&L2_copy, v1, v2);
```

```
printList(&L2_copy);
```

```

    printf("G2 после расщепления вершины %d:\n", v);
    splitVertexList(&L2_copy, v);
    printList(&L2_copy);

    printf("ЗАДАНИЕ 3\n");

    GraphMatrix result;

    printf("Объединение G1 и G2:\n");
    unionGraphs(&M1, &M2, &result);
    printMatrix(&result);

    printf("Пересечение G1 и G2:\n");
    intersectGraphs(&M1, &M2, &result);
    printMatrix(&result);

    printf("Кольцевая сумма G1 и G2:\n");
    ringSumGraphs(&M1, &M2, &result);
    printMatrix(&result);

    printf("ЗАДАНИЕ 4\n");

    printf("Декартово произведение G1 и G2:\n");
    cartesianProduct(&M1, &M2, &result);
    printMatrix(&result);

    freeGraphList(&L1);
    freeGraphList(&L2);

    return 0;
}

```

Результат работы программы:

```
ЗАДАНИЕ 1
Введите количество вершин для графа G1: 5
Введите количество вершин для графа G2: 4
Граф G1 (5 вершин):
Матрица смежности (5 вершин):
  0 1 2 3 4
0 0 0 0 1 1
1 0 0 0 0 0
2 0 0 1 1 1
3 1 0 1 0 0
4 1 0 1 0 0

Граф G2 (4 вершин):
Матрица смежности (4 вершин):
  0 1 2 3
0 1 1 0 1
1 1 0 0 0
2 0 0 0 0
3 1 0 0 1

Список смежности G1:
Список смежности:
Вершина 0: 4 3
Вершина 1:
Вершина 2: 4 3 2
Вершина 3: 2 0
Вершина 4: 2 0

Список смежности G2:
Список смежности:
Вершина 0: 3 1 0
Вершина 1: 0
Вершина 2:
Вершина 3: 3 0

ЗАДАНИЕ 2
Операции над матрицами
Граф G1
Введите вершины для отождествления (v1 v2): 0 1
G1 после отождествления:
```

Вывод: научились реализовывать алгоритм для выполнения унарных и бинарных операций над графами.